

BSS-style Geometric Computation Machine — Concise Spec

One-page implementation blueprint (Three.js + web runtime)

Purpose

A hybrid BSS-inspired engine that ingests bounded streaming geometry (frames, meshes), represents primitives both **discretely** (vertices, edges, triangles, pixels) and **implicitly** (SDF/CSG/predicates), and runs multi-headed agents (heads) with continuous registers to analyze, transform and produce topology- and kinematics-aware outputs under provable resource bounds.

Core components

- **Primitive layers:** Indexed buffers (typed arrays) + Predicate DAG (lazy SDF/CSG nodes).
 - **Attribute (tape) buffers:** per-primitive metadata (labels, parents, symbols).
 - **Heads[]:** bounded array of agents `{pos, orient, logicalIndex, state, type, flags}`.
 - **Adjacency lists:** vertex/face neighbors for traversal.
 - **RingBuffer:** streaming frames + pose metadata.
 - **Quota manager:** `Quota = c * N` enforces linear bound.
-

Head API (logical)

- `read(cell, radius)` → localView
 - `write(cell, value)` (queued)
 - `teleport(vec3)`, `moveAlongNeighbor(k)`, `jumpIndex(idx)`,
`evaluate(predicate, pt)`
 - `proposeEdit(params)`, `score(candidate, window)`
 - Writes resolved deterministically (priority/commutative merge).
-

Tick model (atomic)

1. **Read:** heads sample local attributes.
2. **Decide:** heads compute actions/candidates.
3. **Resolve:** merge concurrent writes deterministically.
4. **Commit:** apply updates to buffer attributes.

Heavy tasks (homology, full materialization) run asynchronously in workers/WASM and merge results on commit.

Primitives & toggles

- **Set algebra:** Union/Intersect/Subtract/Blend on symbolic nodes; lazy evaluation.
 - **Runtime toggles:** EXP, RECURSION_PRIM, MEASURE — change cost semantics; must be explicit in metadata.
 - **Materialization policy:** materialize local indexed patches under quota; cache + LRU eviction.
-

Topological tasks (recommended flow)

- **Quotienting:** in-place union-find in attribute buffers, path-compression by sweeps.
- **Betti numbers:** build discrete simplicial complex (when feasible) → boundary matrix reduction (WASM libs: Ripser/PHAT/GUDHI).
- **Turning/winding/genus:** discrete angle-sum, ray-crossing, Euler characteristic or homology.

Prefer indexed data for topology; use persistence/multiresolution to validate discretization.

Resource & numeric rules

- **Quota enforcement:** every allocation/materialization increments a global counter; refuse or degrade when exceeding Quota.
 - **Simplicial cap:** cap total simplices & simplex dimension (k_max).
 - **Precision:** use quantization/epsilons; avoid floating equality for topology; use interval / higher precision in WASM when needed.
-

Implementation hints

- Use Three.js BufferAttribute for buffers; workers + WASM for heavy homology; GPU shaders for SDF/CSG evaluation and frame differencing; ml5/TF.js in worker for pose detection.
-

Minimal deliverables (starter)

1. Buffer allocator with quota checks.
 2. Head runtime & tick loop.
 3. In-place union-find + adjacency builder.
 4. WASM hook for Ripser/PHAT and worker orchestration.
 5. Debug UI: heads, labels, quota meter.
-

Keep this spec as the single-page implementation blueprint. Use it to guide the initial prototype and enforce linear-bounded resource accounting.